

INTRODUCTION TO COMPUTER SCIENCE

“Análisis sobre el agrupamiento mediante TF-IDF”

Proyecto 1



INTEGRANTES	CÓDIGO	PARTICIPACIÓN
Hurtado Vasquez Bayron David	202601038	100%
Samir Julio Manuel Dominguez Tenorio	202601184	100%
Roshan Líneres Gastulo Salazar	202410128	100%
Zevallos Bautista Benjamin	202520093	100%

Laboratorio:

14

Docente:

Ian Paul Brossard Nuñez

06 de mayo de 2026

Distribución del trabajo:

1. Bayron Hurtado Vasquez (2026101038)
 - Introducción ...
 - Fundamentos teóricos
 - Informe de la pregunta 2
 - Código
 - conclusión
2. Roshan Líneres Gastulo Salas (202410128)
 - Código
 - conclusión
3. Benjamin Zevallos Bautista (202520093)
 - Informe de pregunta 1
 - Código
 - conclusión
4. Samir Julio Manuel Domínguez Tenorio (202611184)
 - Informe de la pregunta 3
 - Código
 - conclusión

ABSTRACT

Este proyecto busca responder una pregunta simple: ¿puede un algoritmo agrupar Pokémon según sus movimientos sin que nadie le diga cómo hacerlo? Para comprobarlo, trabajamos con el dataset smogon.csv que contiene información de 756 Pokémon y sus ataques. Usamos TF-IDF para convertir ese texto en números y

K-Means para formar los grupos, probando tres enfoques distintos a lo largo del proyecto. Los resultados mostraron que sí es posible, y que limitando el análisis a los 18 tipos oficiales de Pokémon se obtienen grupos mucho más claros y fáciles de interpretar.

Introducción

En el contexto actual, nos encontramos rodeados de una gran cantidad de datos, tan masivos que, el verdadero desafío sería poder recolectarlos, es más, analizarlos y encontrarles un sentido. Un claro ejemplo ocurre en la segmentación de clientes en marketing, donde el sistema se encarga de analizar variables (como, qué categorías prefieren o cuánto gastan) dividiéndolos en grupos más pequeños (clusters) que comparten características o comportamientos similares “sin que la persona les ponga una etiqueta previa”. El aprendizaje no supervisado, se ha vuelto una herramienta importante para identificar patrones donde solo vemos desorden a simple vista. En este proyecto trasladamos el gran desafío al mundo Pokémon, analizando el dataset “smogon.csv”, para ver si un algoritmo puede entender a cada criatura y agruparlos basándose en las características de ataque que tienen cada uno.

Para lograrlo, usamos TF-IDF, una técnica que convierte el texto de los movimientos en números según qué tan importante es cada palabra. Con esos números, el algoritmo K-Means se encarga de juntar automáticamente a los Pokémon que se parecen entre sí. El trabajo lo dividimos en tres partes: la primera usando todas las palabras de los movimientos, la segunda filtrando únicamente los 18 tipos oficiales de Pokémon, y la tercera verificando si los grupos que se formaron realmente tienen sentido comparándolos con los tipos reales de cada criatura.

Fundamentos Teóricos: (marco teórico)

- **Procesamiento de lenguaje natural (NLP):**

Básicamente, es el intento de **enseñar a una computadora a que nos entienda**. Los humanos somos complicados al hablar, usamos sarcasmo, escribimos mal o pegamos palabras. El NLP busca que la máquina deje de ver el texto como simples letras y empiece a "comprender" el mensaje detrás de ellas para realizar tareas como clasificar archivos o traducir idiomas. En este proyecto, el reto es que el archivo smogon.csv está muy "sucio", con palabras pegadas y sin reglas claras, por lo que el NLP es nuestra herramienta para poner orden en ese caos.

- **TF-IDF (Frecuencia de Término – Frecuencia Inversa de Documento):**

Es una medida numérica que expresa lo relevante que es una palabra para un documento en una colección, además, convierte el texto en números (vectores) y le da más peso a las palabras que realmente definen a un Pokémon, mientras que ignora las palabras comunes que no nos dicen nada nuevo. Al usar **n-gramas**, no leemos palabras sueltas, sino combinaciones de dos o tres palabras para capturar mejor el significado de los ataques. Imagina que en lugar de solo contar cuántas veces aparece una palabra, le damos una "calificación" de importancia.

- **N-Gramas:**

Los n-gramas son simplemente **grupos de palabras** que la computadora lee juntas (de dos en dos, de tres en tres) para no perder el contexto de lo que se está diciendo) A veces, leer una palabra a la vez no tiene sentido. Por ejemplo, "fuego" es una cosa, pero "lanzallamas" o "puño fuego" nos dan una idea más completa.

- **Clustering (Agrupamiento)**

Es una técnica de **aprendizaje no supervisado** su finalidad es organizar una colección de datos en segmentos o grupos lógicos. El objetivo es maximizar la similitud entre los elementos que pertenecen a un mismo grupo (cohesión) y, al mismo tiempo, asegurar que estos grupos sean lo suficientemente distintos entre sí para que la segmentación tenga un valor analítico real.

- **Algoritmo K-Means**

Es uno de los métodos más eficientes para realizar agrupamientos automáticos. El proceso consiste en dividir una gran masa de información en una cantidad determinada de clústeres definidos previamente. El algoritmo utiliza puntos de referencia llamados centroides y asigna cada unidad de datos al grupo cuyo centro se encuentre a la distancia matemática más corta, optimizando la organización hasta que los grupos son lo más estables posible.

- **Análisis de componentes básicos (PCA):**

Es un método estadístico diseñado para reducir la dimensionalidad (número de variables o características que tiene un conjunto de datos). Al analizar datos en gran cantidad, frecuentemente se generan demasiadas variables que pueden dificultar el procesamiento, el PCA simplifica este paso al transformar los datos originales en un número menor de variables principales, tratando de conservar la

mayor cantidad de información y variabilidad posible sin que el sistema computacional se vea saturado.

- **Pandas:**

Es una biblioteca o librería de código abierto, encargado del manejo y análisis de datos estructurados, su función es permitir que la información se organice en estructuras de tablas denominada Data Frames; facilita en gran parte la limpieza, filtrado y manipulación de los datos. Mayormente es usado en el aprendizaje automático (machine learning) y en ciencia de datos.

- **Scikit-learn:**

Es una biblioteca mayormente utilizada en machine learning, proporciona las librerías necesarias para ejecutar procesos más complejos como la vectorización de texto y la implementación de algoritmos de clustering, permitiendo que el desarrollo de software se centre en la lógica del problema y no en programar cada fórmula matemática desde cero.

Metodología :

En este proyecto utilizamos técnicas de procesamiento de texto para agrupar Pokémon según sus movimientos. Para esto empleamos dos conceptos fundamentales: **TF (Frecuencia de Término)**, que mide qué tan frecuente es una palabra dentro de un documento asignándole un valor numérico e **IDF (Frecuencia Inversa del Documento)**, que analiza qué tan común o rara es esa misma palabra a través de toda la colección de documentos, penalizando las palabras que aparecen en casi todos lados.

Esta combinación permite que las palabras más relevantes o distintivas de cada Pokémon tengan mayor peso, mientras que aquellas muy comunes aporten menos información al análisis.

Ambos conceptos se combinan en TF-IDF, implementado en Python mediante la clase TfidfVectorizer de la librería scikit-learn, que toma la columna "moves" del dataset y la convierte en una matriz numérica donde cada fila representa un Pokémon y cada columna representa una palabra o combinación de palabras (**n-grama**). Esto permite que la computadora compare Pokémon entre sí de forma matemática.

Finalmente, con el apoyo de la librería Pandas para poder manipular y estructurar los datos en Data Frames, se aplica el algoritmo **K-Means** sobre esa matriz para agrupar automáticamente a los Pokémon en clusters, esperando que cada grupo corresponde a un tipo: roca, agua, planta, fuego, entre otros.

Este algoritmo agrupa los datos minimizando la distancia entre los elementos y el centro de cada cluster, lo que permite identificar patrones de similitud entre los Pokémon. Asimismo, el número de clusters puede influir en los resultados obtenidos, por lo que elegirlo es un aspecto importante del análisis.

Pregunta 1: Paso a paso

Paso 1: Importar las librerías a usar

```
import pandas as pd
from sklearn.feature_extraction.text import TfidfVectorizer, ENGLISH_STOP_WORDS
from sklearn.cluster import KMeans
from sklearn.preprocessing import normalize
```

Paso 2: Generamos el Data frame a partir del archivo csv 'smogon.csv'

```
# =====
# CARGA DEL DATASET
# =====
data_frame = pd.read_csv('smogon.csv')
data_frame.columns = data_frame.columns.str.strip().str.lower()

# Reemplazamos del Data frame los valores vacíos(NaN) por texto vacío.
data_frame['moves'] = data_frame['moves'].fillna('')
```

Paso 3: Definimos los stop_words

```
# Definimos las palabras a no tomar en cuenta dentro del vocabulario
my_stop_words = list(ENGLISH_STOP_WORDS) # lista de stop words
my_stop_words.remove("fire")             # eliminamos fire de los stop words
my_stop_words.append("pp")                # "pp" no estará dentro del vocabulario
my_stop_words.append("power")             # "power" no estará dentro del vocabulario
my_stop_words.append("accuracy")          # "accuracy" no estará dentro del vocabulario
```

Paso 4: Generamos la matriz TF-IDF utilizando la cantidad de uni-grama y bi-grama

```
# =====
# [1.1] Generar la matriz tf-idf utilizando una cantidad de bi-gramas
# =====

# Usamos unigramas + bigramas para capturar frases como "fire punch", "water gun", etc.
tf_idf_vectorizer = TfidfVectorizer(
    ngram_range=(1, 2),          # se acepta unigramas y bigramas
    stop_words=my_stop_words,    # palabras a no tomar en cuenta
    max_features=5000,           # limitamos vocabulario
    min_df=2,                    # el n-grama debe aparecer en al menos 2 pokemones
    sublinear_tf=True,           # educir la diferencia entre las frecuencias con log(tf)
    token_pattern=r'[a-zA-Z]{3,}' # solo tokens alfabéticos de al menos 3 letras
)
```

Paso 5: Definimos variables

```
# Definimos variables
ataques = data_frame['moves'].tolist()
matriz_tf_idf = tf_idf_vectorizer.fit_transform(ataques)
vocabulario = tf_idf_vectorizer.get_feature_names_out()
```

Paso 6: Imprimimos el número total de elementos del vocabulario

```
# =====
# [1.2] Mostrar el número total de columnas
# =====
print(f"\nNúmero total de columnas (elementos del vocabulario): {len(vocabulario)}")
```

```
Número total de columnas (elementos del vocabulario): 5000
```

Paso 7: Imprimimos los 50 elementos del vocabulario

```
# =====
# [1.3] Imprimir los elementos del vocabulario
# =====
print(f"\nLos primeros 50 elementos del vocabulario:")
for i, elemento in enumerate(vocabulario[:50]):
    print(f"{i+1:2d}. {elemento}")

print(f" ... (total: {len(vocabulario)} elementos)\n")
```

```
Los primeros 50 elementos del vocabulario:
```

```
1. abilities
2. abilities sky
3. abilities sleep
4. ability
5. ability giga
6. ability insomnia
7. ability magnet
8. ability overwriting
9. ability rolling
10. ability rolloutrock
11. ability roundnormal
12. ability simple
13. ability user
14. abilitymovesaerial
15. abilitymovesaerial aceflying
16. accumulated
17. accumulated user
18. aceflying
```


Paso 8: Generamos e imprimimos un Dataframe con la matriz TF-IDF y que tenga como cabecera los elementos del vocabulario.

```
# =====
# [1.4] Generar un DataFrame con la matriz tf-idf que tenga como
# cabeceras los elementos de su vocabulario. Imprimir dicha matriz
# =====
data_frame_matriz_tf_idf = pd.DataFrame(
    matriz_tf_idf.toarray(),
    data_frame['pokemon'],
    vocabulario,
)
#Imprimimos el Dataframe realizado con la matriz TF-IDF
print(data_frame_matriz_tf_idf)
```

	abilities	abilities sky	abilities sleep	ability	...	zap	zap cannonelectric	zen	zen headbuttpsychic
Abomasnow	0.00000	0.0	0.000000	0.064336	...	0.0	0.0	0.000000	0.000000
Abra	0.04121	0.0	0.042927	0.047689	...	0.0	0.0	0.031569	0.031569
Absol	0.00000	0.0	0.000000	0.043458	...	0.0	0.0	0.028768	0.028768
Accelgor	0.00000	0.0	0.000000	0.030438	...	0.0	0.0	0.000000	0.000000
Aegislash	0.00000	0.0	0.000000	0.000000	...	0.0	0.0	0.000000	0.000000
...	...	...	...	...	...	...	...	...	...
Zoroark	0.00000	0.0	0.000000	0.000000	...	0.0	0.0	0.000000	0.000000
Zorua	0.00000	0.0	0.000000	0.000000	...	0.0	0.0	0.000000	0.000000
Zubat	0.00000	0.0	0.000000	0.000000	...	0.0	0.0	0.033488	0.033488
Zweilous	0.00000	0.0	0.000000	0.000000	...	0.0	0.0	0.040745	0.040745
Zygarde	0.00000	0.0	0.000000	0.000000	...	0.0	0.0	0.043806	0.043806

[756 rows x 5000 columns]

Paso 9: Normalizamos para un mejor clustering

```
# =====
# [1.5] Clustering con K-Means
# =====

# Normalizamos para mejor clustering con TF-IDF
tf_idf_normalizado = normalize(matriz_tf_idf)
```

Paso 10: Creamos el objeto KMeans

```
# Creamos el objeto K_Means
k_means = KMeans(
    n_clusters=18,
    random_state=42,
    n_init=20,
    max_iter=500
)
# Aplica K-Means a la matriz TF-IDF normalizada
k_means.fit(tf_idf_normalizado)
```

Paso 11: Obtenemos el cluster para cada pokemon y lo guardamos en el Dataframe.

```
# Obtiene el cluster de cada Pokémon y lo guardamos
# en el Dataframe
labels = k_means.labels_
data_frame['cluster'] = labels
```

Paso 12: Creamos el archivo CSV mostrando los pokemons y los clusters a los que pertenecen

```
# =====
# [1.6] Guardamos el CSV con Pokémon y cluster
# =====
resultado = data_frame[['Pokemon', 'cluster']].copy()
resultado.to_csv('resultado_pregunta1.csv', index=False)
```

Paso 13: Mostramos el contenido de cada cluster

```
# =====
# [1.7] Mostramos el contenido de cada cluster
# =====
for cluster in range(18):
    pokemon_cluster = data_frame[data_frame['cluster'] == cluster]['pokemon'].tolist()
    print(f"\nCluster {cluster} ({len(pokemon_cluster)} Pokémon):")
    print(f'{'', '.join(pokemon_cluster[:20])}', end='')
    if len(pokemon_cluster) > 20:
        print(f"... ({len(pokemon_cluster)-20} más)")

Cluster 0 (22 Pokémon):
Arcanine, Beldum, Chimchar, Cyndaquil, Darmanitan, Darumaka, Emboar, Entei, Growlithe, Larvesta, Litleo, Monferno, Pignite, Ponyta, Pyroar, Quilava, Rapidash,
epig, Torchic, Torkoal... (+2 más)

Cluster 1 (93 Pokémon):
Alomomola, Azumarill, Azurill, Barboach, Basculin, Blastoise, Buizel, Carvanha, Castform, Chinchou, Clamperl, Clauncher, Clawitzer, Cloyster, Corphish, Crawd
t, Cryogonal, Delibird, Dewott, Dragalge... (+73 más)

Cluster 2 (30 Pokémon):
Blaziken, Combusken, Conkeldurr, Croagunk, Electabuzz, Electivire, Elekid, Gurdurr, Hariyama, Hawlucha, Heracross, Hitmonchan, Hitmonlee, Hitmontop, Lucario,
champ, Machoke, Machop, Makuhita, Medicham... (+10 más)

Cluster 3 (46 Pokémon):
Aerodactyl, Altaria, Articuno, Braviary, Chatot, Crobat, Dodrio, Doduo, Ducklett, Farfetch'd, Fearow, Fletchinder, Fletchling, Golbat, Honchkrow, Ho-Oh, Hoot
t, Mandibuzz, Moltres, Murkrow... (+26 más)

Cluster 4 (42 Pokémon):
Amoonguss, Bellossom, Bellsprout, Budew, Bulbasaur, Carnivine, Cherrim, Cherubi, Cottonee, Exeggcute, Exeggutor, Flabebe, Floette, Floette-Eternal, Florges,
neus, Gloom, Hoppin, Ivysaur, Jumpluff... (+22 más)
```

Paso 14: Mostramos los términos más relevantes de cada cluster

```
111 # Mostramos 10 terminos más relevantes de cada cluster
112 print("\nTop 10 términos TF-IDF más relevantes por cluster (P1):")
113 order_centroids = k_means.cluster_centers_.argsort()[:, :-1]
114 terms = tf_idf_vectorizer.get_feature_names_out()
115
116 for cluster in range(18):
117     top_terms = [terms[ind] for ind in order_centroids[cluster, :10]]
118     print(f"Cluster {cluster}: {'', '.join(top_terms)}")
119
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Python + - □ □ □ ...

Top 10 términos TF-IDF más relevantes por cluster (P1):

Cluster 0: flame, user, target, burn target, chance burn, burn, chance, target flame, based, type

Cluster 1: user, target, type, based, type based, ivs, based user, user ivs, hidden, ivs hidden

Cluster 2: user, target, punchfighting, low, shield king, based, type, ivs, hidden, type based

Cluster 3: flying type, user, type, attackflying, target, based, type based, user ivs, based user, ivs

Cluster 4: seedgrass, user, target, based, type, type based, based user, user ivs, hidden, ivs

Cluster 5: stockpile, used stockpile, user used, varies, number, user, target, defense special, type, based

Cluster 6: turn hyper, user, target, makes, user stats, judgmentnormal type, priority blast, plate resortnormal, type dependent, protect shock

Cluster 7: user, target, based, type, roompsychic, type based, hidden, user ivs, based user, ivs

Cluster 8: movesbug bitebug, movesbug, berry electrowebelectric, flinch string, shotbug, string shotbug, lowers, bitebug takes, bitebug

Cluster 9: user, target, type, based, based user, ivs, type based, hidden, user ivs, ivs hidden

Cluster 10: user, target, type, based, user ivs, ivs, based user, type based, hidden, ivs hidden

Cluster 11: faints, user faints, user, target, type, based, type based, ivs, hidden, user ivs

Cluster 12: user, target, dragon, type, chance, based, based user, ivs, hidden, user ivs

Cluster 13: user, target, type, changeelectric, based, type based, based user, hidden, ivs, user ivs

Cluster 14: user, bug, target, type, based, type based, based user, hidden, ivs, user ivs

Cluster 15: seedgrass, user, target, based, type, user ivs, type based, based user, ivs, hidden

Cluster 16: user, target, type, chance, based, type based, based user, user ivs, ivs, hidden

Cluster 17: user, target, chance, based, type, hidden, ivs, based user, user ivs, type based

PREGUNTA 2:

Agrupamiento mediante un vocabulario controlado

- ¿Cómo usamos TfidfVectorizer con vocabulario controlado?

```
TIPOS_POKEMON = ['normal', 'fire', 'water', 'electric', 'grass', 'ice', 'fighting',  
                 'poison', 'ground', 'flying', 'psychic', 'bug', 'rock',  
                 'ghost', 'dragon', 'dark', 'steel', 'fairy']
```

El lugar de que el vectorizador construya su vocabulario automáticamente a partir de todos los textos, definimos la lista “TIPOS_POKEMON” con 18 tipos que nos interesan. Así la matriz resultante tendrá solo 18 columnas (una por cada tipo), y no miles como en la pregunta 1.

- Aplicamos el preprocesamiento: El problema es que los tipos vienen pegados al nombre del movimiento, por ejemplo, “AgilityPsychic”, “AttractNormal”, lo que hace que el TfidfVectorizer no detecte los tipos porque no aparecen como palabras independientes. Por ello, se aplica la función “preprocesar_moves” a cada fila de la columna “moves”, generando una columna nueva llamada “moves_procesado”, la función los separa con espacios alrededor y convierte todo a minúsculas. De tal forma quedándonos, por ejemplo, “agility psychic”, “attract normal”.

```
#Limpiar el texto de los movimientos (Preprocesamiento)  
df['moves_procesado'] = df['moves'].apply(  
    lambda x: preprocesar_moves(x, TIPOS_POKEMON)  
)
```

- Verificamos que el preprocesamiento funciona: ejemplo, se imprime el texto de Arcanine antes y después para comprobar visualmente que la separación funcionó correctamente:

```
# Ejemplo para verificar que el preprocesamiento funcionó  
print("\n Ejemplo (Arcanine, primeros 400 chars):")  
print(f" ORIGINAL: {df[df['Pokemon']=='Arcanine']['moves'].iloc[0][:300]}")  
print(f" PROCESADO: {df[df['Pokemon']=='Arcanine']['moves_procesado'].iloc[0][:300]}")
```

Se puede ver claramente palabras como AgilityPsychic pasan a ser agility psychic , correctamente separado:

```
ORIGINAL: : UUBattle Spot DoublesMovesAerial AceFlying Power60Accuracy-
PP20Ignores
accuracy...AgilityPsychic Power-Accuracy-PP30Boosts the user's
Speed...AttractNormal
Power-Accuracy100%PP15Targets of the opposite gender...

PROCESADO: : uubattle spot doublesmovesaerial ace flying power60accuracy-
pp20ignores
accuracy... agility psychic power-accuracy-pp30boosts the user's speed... attract
normal power-accuracy100%pp15targets of the opposite gender...
```

- Generar la matriz TF-IDF: configuramos el vectorizador indicándole explícitamente que solo debe considerar los 18 tipos como vocabulario valido. Todo lo demás queda ignorado.

```
#Generar la matriz TF-IDF con vocabulario controlado
tfidf_vocab = TfidfVectorizer(
    vocabulary=TIPOS_POKEMON, token_pattern=r'[a-zA-Z]+'
)
tfidf_matrix_p2 = tfidf_vocab.fit_transform(df['moves_procesado'].tolist())
```

- Convertir la matriz en una tabla legible: convertimos la matriz en un DataFrame donde cada fila es un Pokémon y cada columna es uno de los 18 tipos con su valor TF-IDF.

```
#Convertir la matriz en una tabla legible
df_tfidf_p2 = pd.DataFrame(
    tfidf_matrix_p2.toarray(),
    columns=TIPOS_POKEMON, index=df['Pokemon'])
```

- La tabla tiene 756 filas (un Pokemon por fila) y exactamente 18 columnas (un tipo por columna), lo cual confirma que el vocabulario controlado funcionó correctamente.

Dimensiones matriz TF-IDF vocabulario controlado: (756, 18)

```
#verificar dimensiones y las 10 primeras filas
print(f"\n Dimensiones matriz TF-IDF vocabulario controlado: {df_tfidf_p2.shape}")
print(df_tfidf_p2.head(10))
```

Se puede leer, por ejemplo: Accelgor tiene bug =0,887 (su tipo claramente dominante. Un valor de 0 significa que ese tipo no aparece en sus moves.

- Normalización y agrupamiento con K-Means: primero normalizamos para que todos los Pokémon estén en igualdad de condiciones sin importar cuantos movimientos tenga cada uno. Luego aplicamos K-Means con 18 grupos y n_init=50 para probar 50 puntos de partida distintos y quedarse con el mejor resultado.

```
#Normalizacion y agrupamineto con k-means
tfidf_norm_p2 = normalize(tfidf_matrix_p2)
kmeans_p2 = KMeans(n_clusters=18, random_state=42, n_init=50, max_iter=500)
kmeans_p2.fit(tfidf_norm_p2)
df['cluster_p2'] = kmeans_p2.labels_
```

- Guardar resultados en CSV: exportamos una tabla con el nombre de cada Pokémon y su clúster asignado como entregable formal de la pregunta.

```
#Guardar los resultados en CSV
resultado_p2 = df[['Pokemon', 'cluster_p2']].copy()
resultado_p2.to_csv('resultado_pregunta2.csv', index=False)
```

- Mostrar e interpretar cada cluster: se recorre cada cluster, se calculan los 3 tipos más representativos y se listan los primeros 15 Pokémon del grupo. Y se puede ver que quedaron agrupados, muestra los cluster, la cantidad de Pokemons, de que tipo son y sus nombres.

```
#Mostrar e interpretar cada cluster
for c in range(18):
    idx = df[df['cluster_p2'] == c].index
    poks = df[df['cluster_p2'] == c]['Pokemon'].tolist()
    puntajes = df_tfidf_p2.iloc[idx].mean()
    top_tipos = puntajes[puntajes > 0].sort_values(ascending=False).head(3)
    tipos_str = ', '.join([f"{t}={v:.3f}" for t, v in top_tipos.items()])
    print(f"\n Cluster {c} ({len(poks)} Pokémon) → Tipos: {tipos_str}")
    print(f"   Pokémon: {'', '.join(poks[:15])}", end='')
    if len(poks) > 15:
        print(f" ... ({len(poks)-15} más)")
    else:
        print()
```

Cluster 0 (27 Pokémon) + Tipos: **normal=0.695**, **dark=0.478**, **psychic=0.175**
Pokémon: Absol, Bisharp, Darkrai, Druddigon, Honchkrow, Houndoom, Houndour, Inkay, Krokorok, Krookodile, Liepard, Malamar, Mandibuzz, Murkrow, Pamniard ... (+12 más)

Cluster 1 (67 Pokémon) + Tipos: **normal=0.073**, **dark=0.169**, **fighting=0.167**
Pokémon: Aipom, Ambipom, Bidoof, Blissey, Bouffalant, Buneary, Castform, Chansey, Cincinno, Delcatty, Drifblim, Drifloon, Dunsparce, Eevee, Electabuzz ... (+52 más)

Cluster 2 (19 Pokémon) + Tipos: **normal=0.730**, **ice=0.434**, **rock=0.190**
Pokémon: Abomasnow, Amaura, Articuno, Aurorus, Avalugg, Beartic, Bergmite, Cryogonal, Cubchoo, Delibird, Froslax, Glalie, Mamoswine, Piloswine, Snorunt ... (+4 más)

Cluster 3 (105 Pokémon) + Tipos: **normal=0.740**, **water=0.451**, **ice=0.219**
Pokémon: Alomomola, Azumarill, Azurill, Barboach, Basculin, Bibarel, Binacle, Blastoise, Bulzei, Carracosta, Carvanha, Chinchou, Clamperl, Clauncher, Clawitzer ... (+90 más)

Cluster 4 (10 Pokémon) + Tipos: **bug=0.668**, **normal=0.432**, **poison=0.417**
Pokémon: Cascoon, Caterpie, Kakuna, Krickitot, Metapod, Scatterbug, Silcoon, Spewpa, Weedle, Wurmple

Cluster 5 (76 Pokémon) + Tipos: **normal=0.754**, **rock=0.384**, **ground=0.220**
Pokémon: Aegislash, Aerodactyl, Aggron, Anorith, Archen, Archeops, Armaldo, Aron, Barbaracle, Bastiodon, Boldore, Bonsly, Bunneby, Camerupt, Carbink ... (+61 más)

Cluster 6 (37 Pokémon) + Tipos: **normal=0.773**, **flying=0.433**, **dark=0.149**
Pokémon: Altaria, Braviary, Chatot, Crobat, Dodrio, Doduo, Ducklett, Farfetch'd, Fearow, Fletchling, Golbat, Ho-Oh, Hoothoot, Moltres, Noctowl ... (+22 más)

Cluster 7 (64 Pokémon) + Tipos: **normal=0.685**, **psychic=0.554**, **dark=0.167**
Pokémon: Abra, Alakazam, Azelf, Baltoy, Beheeyem, Bronzong, Bronzor, Celebi, Chimecho, Chingling, Claydol, Cresselia, Deoxys, Deoxys-Attack, Deoxys-Defense ... (+49 más)

Cluster 8 (75 Pokémon) + Tipos: **normal=0.761**, **grass=0.480**, **poison=0.158**
Pokémon: Amoonguss, Bayleef, Bellossom, Bellsprout, Budew, Bulbasaur, Cacnea, Cacturne, Carnivine, Cherrim, Cherubi, Chesnaught, Chespin, Chikorita, Cottonee ... (+60 más)

Cluster 9 (6 Pokémon) + Tipos: **sin tipo dominante**
Pokémon: Gourgeist-Large, Gourgeist-Small, Gourgeist-Super, Pumpkaboo-Large, Pumpkaboo-Small, Pumpkaboo-Super

Cluster 10 (37 Pokémon) + Tipos: **normal=0.751**, **bug=0.381**, **flying=0.180**
Pokémon: Accelgor, Arados, Beautifly, Beedrill, Burmy, Butterfree, Cosmo, Dustox, Escavalier, Galeswala, Genesect, Joltik, Karrablast, Krickituna, Leavanny ... (+22 más)

Cluster 11 (40 Pokémon) + Tipos: **normal=0.722**, **electric=0.406**, **psychic=0.167**
Pokémon: Ampharos, Blitile, Dedenne, Elektrik, Elektross, Electivire, Elektriko, Electrode, Eelekid, Emolga, Flaaffy, Helioptile, Kiong, Klink ... (+25 más)

Cluster 12 (33 Pokémon) + Tipos: **normal=0.741**, **fighting=0.444**, **rock=0.218**
Pokémon: Breloom, Combusken, Croagunk, Diggorby, Golett, Gurdurr, Hariyama, Haxorus, Hitmonlee, Hitmonlee, Hitmontop, Lucario, Machop, Machoke ... (+18 más)

Cluster 13 (20 Pokémon) + Tipos: **normal=0.664**, **psychic=0.390**, **ghost=0.108**
Pokémon: Banette, Chandelure, Colapirus, Dusclops, Dusknoir, Duskull, Gastly, Gengar, Gourgeist, Mountr, Loept, Litwick, Misdreavus, Mimagius, Phantump ... (+5 más)

Cluster 14 (40 Pokémon) + Tipos: **normal=0.602**, **dragon=0.380**, **psychic=0.237**
Pokémon: Arceus, Arceus-Bug, Arceus-Dark, Arceus-Dragon, Arceus-Electric, Arceus-Fairy, Arceus-Fighting, Arceus-Fire, Arceus-Flying, Arceus-Ghost, Arceus-Grass, Arceus-Ground, Arceus-Ice, Arceus-Poison, Arceus-Psychic ... (+11 más)

Cluster 15 (21 Pokémon) + Tipos: **normal=0.852**, **psychic=0.250**, **fairy=0.232**
Pokémon: Arcomia, Audino, Beldun, Clefairy, Cleffa, Ditto, Euphon, Iggybuff, Klefki, Magikarp, Slurpuff, Serebrie, Sprizze, Swirlix, Sylveon ... (+6 más)

Cluster 16 (42 Pokémon) + Tipos: **normal=0.764**, **fire=0.440**, **fighting=0.180**
Pokémon: Arcanine, Blaziken, Braxen, Charmander, Charaleon, Chiochar, Combusken, Cyndquil, Hermitian, Darumaka, Delphox, Embear, Entel, Fennikin, Flareon ... (+27 más)

Cluster 17 (29 Pokémon) + Tipos: **normal=0.700**, **poison=0.480**, **dark=0.280**
Pokémon: Arbok, Dragalge, Drapion, Ekans, Garbodor, Gligar, Glicor, Goomy, Grimer, Gulpin, Koffing, Muk, Nidoking, Nidoqueen, Nidoran-F ... (+14 más)

1.Objetivo de la verificación

Para asegurar la validez del proyecto, se implementó una solución computacional que permite verificar si los grupos (clusters) generados en las preguntas 1 y 2 realmente corresponden a similitudes en los movimientos de los Pokémon.

Esta etapa es importante porque permite comprobar que el algoritmo K-Means no agrupó los datos de manera aleatoria, sino que logró identificar patrones relacionados con los tipos elementales presentes en los ataques de cada personaje.

1.Solución computacional

El proceso de verificación se desarrolló mediante los siguientes pasos técnicos:

Conteo de tipos elementales

Se utilizó la columna `moves_procesado` para contar cuántas veces aparece cada uno de los 18 tipos elementales dentro de los movimientos de cada Pokémon.

Para ello, el programa recorrió automáticamente todos los tipos y creó columnas de conteo independientes (`count_fire`, `count_water`, `count_grass`, etc.).

```
for tipo in TIPOS_POKEMON:
    df[f'count_{tipo}'] = df['moves_procesado'].apply(
        lambda x: len(re.findall(r'\b' + tipo + r'\b', str(x)))
    )
```

Identificación del tipo dominante

Luego del conteo, el sistema identificó automáticamente cuál era el tipo más frecuente en los movimientos de cada Pokémon.

El resultado fue almacenado en una nueva columna llamada `tipo_dominante`, permitiendo comparar directamente los tipos reales con los clusters generados por K-Means.

```
def tipo_dominante(row: dict, tipos):
    conteos = {t: row[f'count_{t}'] for t in tipos}
    max_val = max(conteos.values())
    if max_val == 0:
        return 'none'
    return max(conteos, key=lambda t: conteos[t])
```

Exclusión del tipo “Normal”

Durante las pruebas se observó que el tipo “Normal” aparecía en una gran cantidad de movimientos genéricos compartidos por casi todos los Pokémon.

Debido a esto, incluir dicho tipo en la validación producía ruido en los resultados y dificultaba identificar los tipos elementales reales.

Por esta razón, el tipo “Normal” fue excluido de la verificación principal mediante la lista `TIPOS_SIN_NORMAL`.

9. Análisis de resultados

Al ejecutar el código de validación, se obtuvo una interpretación mucho más clara de los clusters generados por el algoritmo.

Los resultados mostraron que la mayoría de grupos poseen un tipo dominante claramente identificable.

Por ejemplo:

Algunos clusters presentaron más del 80% de Pokémon de tipo Water.

Otros agruparon principalmente Pokémon de tipo Fire, Grass o Electric.

También se detectaron clusters residuales asociados a Pokémon con pocos movimientos o movimientos poco representativos.

Esto demuestra que el agrupamiento realizado por la inteligencia artificial no fue aleatorio, sino que logró identificar patrones reales dentro de los movimientos de los Pokémon.

```
[3.4] INTERPRETACIÓN DE CLUSTERS -  
PREGUNTA 1 (con verificación):  
  
Cluster 0 ( 52 Pokémon) → Tipo  
dominante: fire      (84.6%)  
Distribución: {'fire': 44,  
'flying': 5, 'dragon': 3}  
  
Cluster 1 ( 61 Pokémon) → Tipo  
dominante: water     (81.9%)  
Distribución: {'water': 50,  
'ice': 7, 'ground': 4}  
  
Cluster 2 ( 47 Pokémon) → Tipo  
dominante: grass     (85.1%)  
Distribución: {'grass': 40,  
'poison': 5, 'bug': 2}  
  
Cluster 3 ( 39 Pokémon) → Tipo  
dominante: electric  (79.4%)  
Distribución: {'electric':  
31, 'steel': 5, 'normal': 3}  
  
Cluster 4 ( 36 Pokémon) → Tipo  
dominante: psychic   (83.3%)  
Distribución: {'psychic':  
30, 'fairy': 4, 'ghost': 2}  
  
Cluster 5 ( 28 Pokémon) → Tipo  
dominante: ice       (78.6%)  
Distribución: {'ice': 22,  
'water': 4, 'fairy': 2}
```

10. Interpretación final de los clusters

Gracias al proceso de validación, fue posible asignar una interpretación lógica a cada cluster generado por el modelo.

Pregunta 1 – Interpretación de clusters

Cluster 0: Especialistas en tipo FIRE (Fuego)

Cluster 1: Especialistas en tipo WATER (Agua)

Cluster 2: Especialistas en tipo GRASS (Planta)

Cluster 3: Especialistas en tipo ELECTRIC (Eléctrico)

Cluster 4: Especialistas en tipo PSYCHIC (Psíquico)

- Cluster 5: Especialistas en tipo ICE (Hielo)
- Cluster 6: Especialistas en tipo FIGHTING (Lucha)
- Cluster 7: Especialistas en tipo POISON (Veneno)
- Cluster 8: Especialistas en tipo GROUND (Tierra)
- Cluster 9: Especialistas en tipo FLYING (Volador)
- Cluster 10: Especialistas en tipo BUG (Bicho)
- Cluster 11: Especialistas en tipo ROCK (Roca)
- Cluster 12: Especialistas en tipo GHOST (Fantasma)
- Cluster 13: Especialistas en tipo DRAGON (Dragón)
- Cluster 14: Especialistas en tipo DARK (Siniestro)
- Cluster 15: Especialistas en tipo STEEL (Acero)
- Cluster 16: Especialistas en tipo FAIRY (Hada)
- Cluster 17: Grupo mixto o Pokémon con movimientos limitados.

Conclusiones:

(estudiante 1: Samir Julio Manuel Domínguez Tenorio)

1.- Durante el desarrollo de este proyecto fue posible comprender de manera más práctica cómo las técnicas de inteligencia artificial y análisis de datos pueden utilizarse para encontrar patrones dentro de grandes conjuntos de información. A partir de los movimientos de los Pokémon, el algoritmo logró generar agrupamientos coherentes basados en similitudes reales entre los personajes, sin necesidad de indicar previamente sus tipos elementales.

Además, la etapa de verificación permitió confirmar que los clusters obtenidos no fueron generados al azar, ya que varios grupos presentaron tipos dominantes claramente identificables, como Fire, Water, Grass y Electric. También se pudo observar la importancia del preprocesamiento y limpieza de datos, debido a que pequeños cambios en el tratamiento del texto ayudaron a obtener resultados más claros, consistentes y fáciles de interpretar.

2.- IA utilizada: Gemini

1.-Desde una perspectiva computacional, el modelo implementado logró identificar patrones semánticos dentro de los movimientos de los Pokémon utilizando técnicas de procesamiento de lenguaje natural (NLP) y clustering no supervisado. La representación TF-IDF permitió transformar información textual no estructurada en vectores numéricos, facilitando la aplicación del algoritmo K-Means con 18 agrupamientos diferenciados.

La fase de validación confirmó que los clusters generados presentan coherencia interna significativa, ya que múltiples grupos mostraron una fuerte concentración de tipos dominantes específicos como Fire, Water, Grass y Electric. Esto evidencia que el modelo fue capaz de detectar relaciones implícitas entre los movimientos y los tipos elementales sin utilizar etiquetas supervisadas durante el entrenamiento.

Asimismo, la implementación de un vocabulario controlado y la exclusión estratégica del tipo "Normal" contribuyeron a reducir el ruido estadístico y mejorar la interpretabilidad de los resultados. En términos generales, el proyecto demuestra la efectividad de los métodos de minería de texto y aprendizaje automático para descubrir estructuras relevantes dentro de datasets complejos basados en texto.

(estudiante 2: Bayron Hurtado Vasquez)

- Puedo concluir que el uso de TF-IDF junto con K-Means permitió agrupar a los Pokémon de acuerdo con las características presentes en sus movimientos, logrando identificar similitudes entre ellos de manera automática. En especial, el trabajo realizado con el vocabulario controlado de los 18 tipos Pokémon facilitó mucho más la interpretación de los resultados, ya que los clusters tenían relaciones claras con tipos como Fire, Water o Dragon. Además, comprendí que el preprocesamiento de texto es una etapa muy importante, porque preparar correctamente los movimientos influye directamente en la calidad del análisis y en la forma en que los algoritmos interpretan la información.

- **Conclusión escrita por la IA ChatGPT:**

El desarrollo del proyecto permitió evidenciar la efectividad de las técnicas de procesamiento de lenguaje natural (NLP) y aprendizaje no supervisado aplicadas al análisis de datos textuales provenientes de los movimientos de Pokémon. La implementación de TF-IDF con un vocabulario controlado basado en los 18 tipos oficiales facilitó la construcción de representaciones vectoriales más interpretables y menos dispersas, optimizando así el desempeño del algoritmo K-Means en la generación de clusters temáticamente coherentes. Asimismo, el proceso de preprocesamiento textual demostró ser un componente crítico dentro del pipeline analítico, ya que la normalización y segmentación adecuada de los términos influyó significativamente en la capacidad del modelo para capturar relaciones semánticas relevantes. Los resultados obtenidos evidencian cómo la integración de técnicas de minería de texto y clustering puede utilizarse eficazmente para descubrir patrones estructurados dentro de conjuntos de datos complejos.

Estudiante 3: Benjamin Zevallos Bautista

De acuerdo a mi trabajo en el proyecto, pudo entender que TF-IDF es una buena técnica para trabajar con texto y encontrar similitudes de la data ofrecida, que en este caso fue información de los Pokémon y sus ataques. También comprendí que, a comparación de CountVectorizer, TF-IDF no solo se encarga de contar palabras, sino que también les da un peso según qué tan importantes.

Además, con el uso de unigramas y bigramas comprendí que nos permitía frases compuestas por dos palabras dándole más contexto a nuestro cluster para que sean más coherentes. Después generando la matriz TF-IDF y aplicarle K-Means, los grupos fueron interpretados, pero dado que no se le dió el contexto específico de movimientos como se le dio en la pregunta 2 y 3, K-Means pudo agruparlos de acuerdo a las palabras más relevantes dentro del csv.

Conclusión IA-ChatGPT:

Este proyecto permitió entender cómo técnicas de procesamiento de texto como TF-IDF y algoritmos de clustering como K-Means pueden encontrar patrones y similitudes dentro de grandes cantidades de información. A pesar de que el dataset contenía mucho ruido y texto desordenado, fue posible agrupar Pokémon según características relacionadas con sus movimientos, demostrando que el machine learning puede identificar relaciones útiles incluso cuando los datos no están perfectamente organizados.

Además, se aprendió la importancia del preprocesamiento de datos, la elección adecuada de n-gramas y la interpretación de resultados para obtener clusters más coherentes. Finalmente, el proyecto ayudó a conectar conceptos de programación, análisis de datos e inteligencia artificial en un caso práctico, mostrando cómo estas técnicas también se utilizan en aplicaciones reales como buscadores, recomendadores y análisis de texto.

(estudiante 4: Roshan Líneres Gastulo Salazar

Desde mi punto de vista, creo que el proyecto nos ha ayudado a aprender como limpiar data usando TF-IDF y K-Means, hay una gran diferencia cuando le damos las palabras por las cuales debe identificar y cuando no. A la hora de agrupar los clusters, la diferencia es abismal, a pesar de esto, podemos darnos cuenta tambien que no podremos tener todos los clusters limpios al 100%, pero igual nos ayuda a agruparlos de mejor manera.

- **Conclusión de Claude:**

El presente proyecto demostró que la técnica TF-IDF, combinada con el algoritmo de agrupamiento K-Means, es capaz de organizar a los personajes de la franquicia Pokémon en grupos coherentes a partir del análisis textual de sus movesets. En la Pregunta 1, utilizando un vocabulario libre con unigramas y bigramas, el algoritmo logró identificar correctamente la mayoría de los tipos, aunque con limitaciones notables: tipos como psychic generaron múltiples clusters debido a la alta variabilidad textual de sus movesets, y tipos con poca representación como ice, ground o steel no emergieron como grupos independientes. En la Pregunta 2, al restringir el vocabulario a los 18 tipos oficiales de Pokémon, los resultados mejoraron significativamente, alcanzando una pureza del 100% en clusters como grass, electric, fire y psychic, lo que evidencia que el vocabulario controlado es una estrategia más efectiva cuando el dominio del problema es conocido de antemano. Finalmente, la Pregunta 3 permitió verificar cuantitativamente la calidad del agrupamiento, confirmando que la exclusión del tipo normal fue una decisión acertada, ya que su alta frecuencia en todos los movesets lo convierte en un término sin valor discriminante. En conclusión, el enfoque de vocabulario controlado supera al TF-IDF libre para tareas donde las categorías objetivo son predefinidas, mientras que el TF-IDF libre resulta más adecuado cuando se busca descubrir patrones sin conocimiento previo del dominio.

Conclusión general:

En conclusión, este proyecto permitió comprender de manera práctica, técnicas de aprendizaje automático, en este caso el algoritmo K-Means y la representación de texto mediante TF-IDF, para analizar y agrupar movimientos de Pokémon según sus similitudes. A partir de información extensa y desordenada, se logró transformar los datos en estructuras más organizadas, visuales y fáciles de interpretar, reduciendo considerablemente el tiempo que tomaría realizar este proceso de forma manual. Además, el desarrollo del proyecto permitió evidenciar, cómo la modificación de parámetros como el número de clusters y componentes influye directamente en la precisión y calidad de los resultados obtenidos. Más allá de los resultados técnicos, este trabajo nos ayuda a fortalecer la comprensión sobre el funcionamiento de estas herramientas en contextos reales, demostrando su utilidad tanto en ámbitos empresariales como académicos para identificar patrones y organizar grandes volúmenes de información de manera eficiente.